

Quiz de Culture Générale

Présentation technique du projet

Antoine Aubry

HEIG-VD

10 juin 2026

- Application web de quiz de culture générale (SPA)
- L'utilisateur choisit une **catégorie** et une **difficulté**
- 10 questions à choix multiple, **temps limité** par question
- Score final + **récapitulatif détaillé** des réponses
- Personnalisation : **thème** clair/sombre, **langue** FR/EN, support **manette**

Architecture globale

```
Utilisateur <-> React (SPA) <-> Open Trivia DB (API REST)
      |
      +-- localStorage (theme, langue)
```

- Application 100% **côté client** (pas de backend propre)
- Toutes les données viennent d'une API publique externe
- Les préférences (thème/langue) sont persistées localement

Qu'est-ce que c'est ? Bibliothèque JavaScript pour construire des interfaces utilisateur basées sur des composants.

Pourquoi ce choix ?

- Très répandu, large écosystème et documentation
- Modèle par composants : découpage naturel (Quiz, QuestionCard, Setup, Home...)
- Hooks (`useState`, `useEffect`, `useContext`) pour gérer état et effets de bord

Avantages

- Re-rendu efficace via le DOM virtuel
- Composants réutilisables et testables
- Communauté énorme, beaucoup de ressources

Inconvénients

- Courbe d'apprentissage (JSX, hooks, cycle de vie)
- Beaucoup de « boilerplate » pour des apps simples
- Pièges classiques : dépendances de `useEffect`, *stale state*

Ce que j'ai appris : la gestion fine des `useEffect` (ex. bug du timer où un effet lisait un état « périmé » lors du changement de question).

Qu'est-ce que c'est ? Librairie de routage côté client pour les SPA React.

Pourquoi ce choix ?

- Standard de facto pour React
- Permet 3 pages distinctes (/, /setup, /quiz) sans rechargement
- `useSearchParams` : transmettre catégorie/difficulté via l'URL
- `useNavigate` : redirections programmatiques (rejouer, changer de paramètres)

Avantages

- Navigation fluide, état conservé
- Paramètres d'URL = configuration partageable

Inconvénients

- API qui a beaucoup changé entre versions
- Concepts supplémentaires à maîtriser (Routes, Outlet...)

Qu'est-ce que c'est ? Client HTTP basé sur les promesses, pour appeler des API REST.

Pourquoi ce choix ?

- Syntaxe simple (`axios.get(url).then(...)`)
- Gestion d'erreurs centralisée (`.catch`)
- Alternative plus pratique que `fetch` natif (pas de `.json()` manuel, gestion des erreurs HTTP)

Avantages

- Code lisible et concis
- Gestion fine des codes d'erreur (ex. 429)

Inconvénients

- Dépendance externe pour une fonctionnalité que `fetch` couvre presque
- Poids supplémentaire dans le bundle

Qu'est-ce que c'est ? API REST publique et gratuite fournissant des questions de quiz par catégorie/difficulté.

Pourquoi ce choix ?

- Gratuite, sans clé API, simple d'usage
- Permet de filtrer par category, difficulty, type
- Liste des catégories disponible via `api_category.php`

Exemple d'appel :

```
GET https://opentdb.com/api.php?amount=10&type=multiple  
&category=9&difficulty=easy
```

Open Trivia DB : avantages / inconvénients

Avantages

- Aucune authentification requise
- Données prêtes à l'emploi (questions, réponses, mauvaises réponses)
- Filtrage simple par paramètres d'URL

Inconvénients

- **Rate limiting** agressif (HTTP 429)
- Texte encodé en entités HTML (", &...)
- Contenu uniquement en **anglais**
- Pas de garantie de disponibilité (service tiers)

Ce que j'ai appris : gérer les erreurs réseau (429), décoder le HTML reçu, et limiter le nombre d'appels (cache via `window.__quizData`).

Context API (React)

Qu'est-ce que c'est ? Mécanisme natif de React pour partager un état global sans « prop drilling ».

Pourquoi ce choix ?

- Besoin de partager le **thème** et la **langue** dans toute l'app
- Pas besoin d'une librairie externe (Redux, Zustand...) pour 2 valeurs simples
- Couplé à `localStorage` pour persister les préférences

Avantages

- Intégré à React, zéro dépendance
- API simple : `Provider` + `useContext`

Inconvénients

- Re-render de tous les consommateurs au changement
- Devient difficile à maintenir si l'état global grossit

Variables CSS pour le mode sombre

Approche : variables CSS (`--bg-color`, `--text-color`, `--accent-color`...) redéfinies sous `[data-theme="dark"]`.

```
:root { --bg-color: #f0f4ff; --text-color: #2c3e50; }  
[data-theme="dark"] { --bg-color: #1a1d24; --text-color: #f0f0f0; }
```

Avantages

- Pas de librairie CSS-in-JS
- Changement de thème instantané (1 attribut HTML)

Inconvénients

- Nécessite de revoir toutes les couleurs codées en dur
- Pas de typage / autocomplétion

Approche : dictionnaire JS (`translations.js`) + fonction `t(cle)` via `LanguageContext`.

Avantages

- Très léger, aucune dépendance (vs. `i18next`)
- Suffisant pour 2 langues et peu de textes

Inconvénients

- Pas de pluriels, formats de dates, etc.
- Le contenu de l'API (questions) reste en anglais

Ce que j'ai appris : structurer des clés imbriquées (`quiz.scorePerfect`) et garder la traduction synchronisée avec l'UI.

Qu'est-ce que c'est ? API web native (`navigator.getGamepads()`) pour lire l'état des manettes connectées.

Pourquoi ce choix ?

- Aucune librairie nécessaire, support natif des navigateurs
- Permet de naviguer/sélectionner les réponses sans clavier/souris

Avantages

- Expérience originale et ludique
- API simple, basée sur du *polling*

Inconvénients

- Pas d'événements : il faut *poller* via `requestAnimationFrame`
- Mappings de boutons variables selon les manettes
- Gestion du « cooldown » pour éviter les répétitions

Ce que j'ai appris

- Structurer une app React en pages / composants / contextes
- Gérer un état complexe (score, timer, historique) avec des hooks
- Déboguer des effets de bord et des problèmes de *stale state*
- Consommer une API REST tierce avec ses contraintes (rate limiting, encodage)
- Mettre en place un système de thème et d'internationalisation léger, sans librairie lourde
- Utiliser une API navigateur peu courante (Gamepad)

- Authentification (Firebase Auth)
- Sauvegarde des scores / classement (Firestore)
- Mode multijoueur, sons, animations

Merci pour votre attention !